

# **Jegyzőkönyv**

## **Párhuzamos eszközök programozása Féléves feladat**

**Párhuzamos Képfeldolgozás: CPU és GPU Architektúrák  
Teljesítményének Összehasonlítása**

Készítette: Molnár Márk

Neptun-kód: TLZ12Y

Miskolc, 2026

## TARTALOMJEGYZÉK

|                                                                             |  |
|-----------------------------------------------------------------------------|--|
| <b>1. Bevezetés.....</b>                                                    |  |
| <b>1.1. A problémafelvetés és a projekt célja.....</b>                      |  |
| <b>1.2. A vizsgált számítási feladat: Szürkeárnyaltos képkonverzió.....</b> |  |
| <b>2. Alkalmazott Technológiák és Architektúrák</b>                         |  |
| <b>2.1. Párhuzamosítás CPU-n: Az OpenMP keretrendszer</b>                   |  |
| <b>2.2. Heterogén feldolgozás GPU-n: Az OpenCL szabvány</b>                 |  |
| <b>2.3. A C program és a mérési szkript felépítése</b>                      |  |
| <b>3. Metodológia és Mérési Környezet</b>                                   |  |
| <b>3.1. Tesztadatok: A feldolgozott képek felbontásai</b>                   |  |
| <b>3.2. A "Fair" mérés elve: Számítási és adatmozgatási idők</b>            |  |
| <b>4. Memóriakezelési Optimalizáció a Videókártyán</b>                      |  |
| <b>4.1. Az alap (1 dimenziós) OpenCL implementáció korlátai</b>             |  |
| <b>4.2. Térbeli lokalitás (Spatial Locality) és a gyorsítótár szerepe</b>   |  |
| <b>4.3. Optimalizált (2 dimenziós) Work-group csempézés megvalósítása</b>   |  |
| <b>5. Eredmények és Kiértékelés</b>                                         |  |
| <b>5.1. A 3-vonalas teljesítménygrafikon bemutatása</b>                     |  |
| <b>5.2. Kis felbontású tartomány: A CPU és a PCIe busz viszonya</b>         |  |
| <b>5.3. Nagy felbontású tartomány: A GPU nyers számítási ereje</b>          |  |
| <b>5.4. A metszéspont elemzése: Az 1D vs. 2D architektúra különbségei</b>   |  |
| <b>6. Összegzés és Konklúzió</b>                                            |  |

# Párhuzamos Képfeldolgozás: CPU és GPU Architektúrák Teljesítményének Összehasonlítása

## 1. Bevezetés

- **A probléma:** A mai világban a digitális képek mérete drasztikusan megnőtt (akár több tíz megapixel). Ezeknek a feldolgozása egyetlen processzormagon (szekvenciálisan) rendkívül lassú.
- **A megoldás:** Párhuzamosítás. A feladat célja egy szürkeárnyaltos (grayscale) konverziós algoritmus implementálása és teljesítménymérése két különböző architektúrán: többmagos processzoron (CPU) és videokártyán (GPU).
- **A kutatási kérdés:** Nem csak azt vizsgáljuk, hogy "melyik a gyorsabb", hanem azt is, hogy *milyen képméretnél és milyen memóriakezelési stratégia mellett éri meg a videokártyát használni.*



## 2. Alkalmazott Technológiák

Itt mutasd be röviden a két technológiát, amit használtál.

- **OpenMP (Processzor):** Közös memóriájú (shared memory) architektúrákhoz fejlesztett API. A projektben a `#pragma omp parallel for` direktívát használtuk, amely a kép pixeleit egyenlő arányban szétosztja a CPU fizikai és logikai magjai között. Előnye, hogy az adatok már eleve a rendszermemóriában (RAM) vannak, így nincs adatmozgatási késleltetés (overhead).

```
// Az OpenMP direktíva, amely szétosztja a pixeleket a
processzormagok között
#pragma omp parallel for
for (int i = 0; i < width * height; i++) {
    int idx = i * channels;

    // Színek kiolvasása
    unsigned char r = image[idx];
```

```

    unsigned char g = image[idx + 1];
    unsigned char b = image[idx + 2];

    // Szürkeárnyalat kiszámítása szabványos súlyozással
    unsigned char gray = (unsigned char)(0.299f * r + 0.587f * g
+ 0.114f * b);

    // Eredmény visszaírása
    image[idx] = gray;
    image[idx + 1] = gray;
    image[idx + 2] = gray;
}

```

- **OpenCL (Videókártya):** Heterogén rendszerek programozására szolgáló keretrendszer. A CPU "gazdaként" (host) működik, és átküldi a feladatot az OpenCL-t támogató grafikus kártyának (device).
  - *Extra pont a tanárnál:* Emeld ki, hogy az OpenCL esetében a programnak gondoskodnia kell a RAM és a VRAM (videómemória) közötti explicit adatmozgatásról (PCIe buszon keresztül), ami komoly szűk keresztmetszet lehet.

### 3. Metodológia és Megvalósítás

Itt írd le, hogy pontosan mit és hogyan mértél. Ez bizonyítja, hogy a mérésed "fair" volt.

- **Az algoritmus:** A konverzió a szabványos súlyozott RGB csatorna átlagolást használja:  $Szürke = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ .
- **Mérési környezet:** A program 7 különböző felbontású képet dolgoz fel, a ~50 ezer pixeles (bélyegkép) mérettől egészen a gigantikus, 45 Megapixeles méretig. A méréseket egy automatizált Batch szkript ötször (5x) hajtja végre a mérési hibák és az operációs rendszeri háttérfolyamatok kizárása érdekében.
- **A Fair Mérés:** Írd le, hogy a CPU és a GPU mérése igazságos volt. A GPU idejébe (stopperóra) *beleszámítottad* a kép átmásolását a RAM-ból a VRAM-ba, a kernel lefutását, és a kész kép visszamásolását is.

```

// =====
// STOPPERÓRA INDÍTÁSA
// =====
double start_time = omp_get_wtime();

// 1. Lépés: Kép másolása a gépből (RAM) a GPU-ra (VRAM)
clEnqueueWriteBuffer(command_queue, image_mem_obj, CL_TRUE, 0,
data_size, image, 0, NULL, NULL);

// Kernel futtatása a GPU-n (2D vagy 1D módban)
clEnqueueNDRangeKernel(command_queue, kernel, ...);

```

```

clFinish(command_queue); // Megvárjuk a számítás végét

// 2. Lépés: Kész kép visszamásolása a GPU-ról (VRAM) a gépre (RAM)
clEnqueueReadBuffer(command_queue, result_mem_obj, CL_TRUE, 0,
data_size, image, 0, NULL, NULL);

// =====
// STOPPERÓRA LEÁLLÍTÁSA
// =====
double end_time = omp_get_wtime();

```

#### 4. Memóriakezelési stratégiák a GPU-n (A Titkos Fegyvered)

Ez az a rész, amivel kiemelkedsz a többi hallgató közül. Külön alfejezetet érdemel!

- Írd le, hogy az OpenCL implementációt kétféleképpen is elkészítetted, hogy vizsgáld a hardveres gyorsítótár (cache) viselkedését.
- **1D Architektúra (Lineáris):** A GPU egyetlen hosszú, egydimenziós tömbként (szalagként) tekint a képre.
- **2D Architektúra (Csempézés / Térbeli lokalitás):** A GPU egy 2 dimenziós (X, Y) rácsként kapja meg az adatokat. Ilyenkor a hardveres Work-group kiosztás térben egymáshoz közeli pixeleket dolgoz fel egyszerre. Ennek köszönhetően maximálisan kihasználja a GPU L1/L2 gyorsítótárát (Spatial Locality), hiszen a beolvasott pixelek szomszédjai már a leggyorsabb memóriában várják a feldolgozást.

```

__kernel void grayscale_2d(__global const unsigned char* input,
                           __global unsigned char* output,
                           const int width,
                           const int channels) {
    // A szálak térbeli (2D) azonosítójának lekérése
    int x = get_global_id(0); // Oszlop (X koordináta)
    int y = get_global_id(1); // Sor (Y koordináta)

    // A 2D-s koordináta átszámítása a lineáris memóriacímre
    int idx = (y * width + x) * channels;

    // ... szürkeárnyaltos matematikai művelet az 1D-hez
    hasonlóan ...
}

```

A C program (Host) oldalán pedig jelezniünk kellett az OpenCL felé, hogy a feldolgozási rács (NDRange) kétdimenziós.

```

// A feldolgozási tér megadása 2 dimenzióban (Szélesség x Magasság)
size_t global_item_size[2] = {width, height};

```

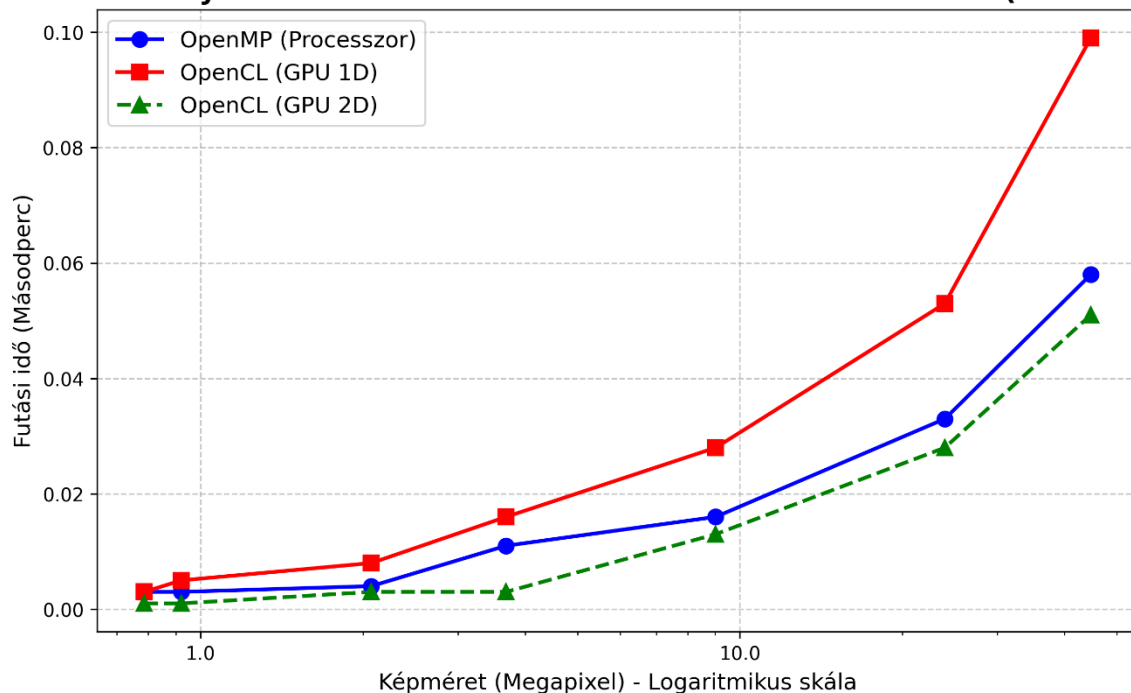
```
ret = clEnqueueNDRangeKernel(command_queue, kernel, 2, NULL,
global_item_size, NULL, 0, NULL, NULL);
```

## 5. Eredmények és Kiértékelés (A Grafikon Elemzése)

Ide illeszd be a Python által generált 3 vonalas (kék, piros, zöld) grafikont! A szövegben elemezd a görbéket három különböző tartományra bontva:

1. **Kis képek tartománya (A grafikon bal oldala):** Látható, hogy a nagyon kicsi képeknél a CPU (kék vonal) a gyorsabb. *Magyarázat:* A videokártya inicializálása és az adat átmásolása a PCIe kábelen több időt vesz igénybe, mint maga a számítás. Kis adatnál a GPU nem tudja kifutni magát.
2. **Közepes képek tartománya (A grafikon közepe):** Ahogy nő a felbontás, a GPU-k (piros és zöld) brutálisan megelőzik a CPU-t. *Magyarázat:* Itt már annyi a számítási feladat (több millió szorzás), hogy a CPU magjai elfogynak, míg a GPU több ezer apró magja játszi könnyedséggel, párhuzamosan darálja le a pixeleket. A számítási nyereség bőségesen kompenzálja a másolási időt.
3. **Extrém nagy képek és a Metszéspont (A grafikon jobb szélé):** Ezen a ponton elemezd a vonalak szétválását! A 45 Megapixeles képnél az **1D-s OpenCL (piros vonal)** meredeken kilő, és akár a CPU-nál is lassabb lehet. Ennek oka a PCI Express busz szűk keresztmetszete és a folyamatos "cache-miss" (gyorsítótár-tévesztés) jelenség a nem optimális memórialepdelés miatt. Ezzel szemben a **2D-s OpenCL (zöld vonal)** gyönyörűen, laposan alatta marad mindkettőnek. A térbeli lokáltság (spatial locality) és az optimális cache-használat miatt a videokártya még a gigantikus adatmozgatási teher ellenére is abszolút győztes tud maradni.

### Szürkeárnyaltos konverzió: Architektúrák összehasonlítása (Mérés: 4/5)



## 6. Konklúzió / Összegzés

Rövid lezárás.

- Bebizonyosodott, hogy a GPU hatalmas gyorsulást hozhat a képfeldolgozásban, de **nem minden esetben**.
- Kis feladatoknál a CPU használata a célravezetőbb az adatmozgatási overhead miatt.
- Ha GPU-t használunk, a nyers kódolás (1D) nem elegendő: a hardver felépítésének (memória architektúra) megfelelő, 2D-s csempézett algoritmusok használata kritikus fontosságú a valós teljesítménynövekedés eléréséhez.